FarmLife ERP

Complete Dairy Management System — Architecture & Development Plan

Business type	Platform	Scale
Complete Dairy ERP	Responsive web application	Multi-farm / multi-branch

A production-ready bilingual (English and Bangla) system for cattle and dairy farm operations, milk collection and distribution, inventory, accounting, reporting, and AI-assisted analytics in Bangladesh.

1. System Architecture

Web frontend (React + TypeScript + Vite + Tailwind) → secure services (Supabase Edge Functions / PostgreSQL functions) → PostgreSQL with farm-aware Row Level Security, Supabase Auth, Storage, audit logs, and backups. Authorized organization-level views consolidate multiple farms and branches.

Layer	Recommended technology
Frontend	React, TypeScript, Vite
UI	Tailwind CSS, shadcn/ui
Backend / Database	Supabase Edge Functions, PostgreSQL
Identity / Security	Supabase Auth, RBAC, Row Level Security
Forms / Data	React Hook Form, Zod, TanStack Query
Charts / Localization	Recharts, i18next (English / Bangla)
Testing / Delivery	Vitest, Playwright, GitHub, Codex, Vercel or Docker

2. Complete Functional Modules

#	Module	Key features
01	Authentication & Roles	Login, invitations, memberships, RBAC and access control
02	Organization & Branches	Multiple farms, sheds, pens, transfers and consolidated views
03	Dashboard	KPIs, production, income, expenses, date comparisons
04	Cattle Registry	IDs, breed, age, photo, parentage and status
05	Lifetime Animal Records	Event timeline from birth/acquisition to sale/death
06	Milk Production	Milking sessions, quality and allocation reconciliation
07	Farmer Milk Collection	Farmers, fat/SNF, rates and settlements
08	Feed & Nutrition	Rations, consumption, costs and plans
09	Health & Veterinary	Vaccines, treatments, follow-ups and withdrawal periods
10	Breeding & Calving	Heat, insemination, pregnancy and calves
11	Calf Growth & Weight	Weights, ADG and growth charts
12	Fattening Management	Cycles, costs, weight gain and realized profit
13	Inventory & Stock	Items, batches, movements, expiry and alerts
14	Purchasing & Suppliers	Orders, receipts, vendors and dues
15	Sales & Customers	Invoices, payments, receivables and animal sales
16	Milk Delivery & Subscriptions	Routes, recurring orders, returns and billing
17	Finance & Accounting	Double-entry ledger, cash flow, P&L and balance sheet
18	Employees & Tasks	Staff, attendance, shifts, payroll and assignments
19	Reports & Analytics	Farm and multi-branch operational/financial reports
20	Notifications	Stock, health, payment and task alerts
21	AI Farm Assistant	Bilingual grounded answers and anomaly summaries
22	Administration & Audit	Settings, audit trails, backups and exports

3. Multi-Farm Requirements

- Organizations own multiple farms, collection centers and distribution branches.
- Farm-specific cattle, milk, inventory, workers and transactions are isolated by authorization.
- Owners can access authorized consolidated reporting; managers only assigned locations.
- Cross-farm transfers require traceable source, destination and approval records.
- Internal transfers must not inflate external revenue or consolidated profit.

4. User Roles & Permissions

Role	Primary access
Owner / Super Admin	All authorized farms, configuration, financial records, users and reports
Organization Administrator	Organization configuration and delegated branch management
Farm Manager	Cattle, milk, feed, breeding, health schedules and operations
Collection Manager	Farmers, collection quality, pricing and settlements
Accountant	Invoices, payments, journals and finance reports
Veterinarian	Exams, prescriptions, vaccinations and follow-ups
Farm Worker	Assigned feeding, milking, weight and task records
Delivery Staff	Assigned deliveries, returns and payment collection
Report Viewer / Auditor	Read-only access to specifically permitted data

Permissions must be enforced in the backend and database, not only by hiding UI controls.

5. Database Structure

Domain	Proposed tables
Organization	organizations, farms, branches, sheds, pens
Identity	profiles, memberships, roles, role_permissions
Cattle	animals, animal_events, animal_transfers, animal_relationships
Milk	milking_sessions, milk_records, milk_allocations, milk_batches
Collection	farmers, collection_records, milk_quality_tests, farmer_settlements
Feed	feed_items, ration_templates, feed_consumptions
Health	health_records, treatments, vaccinations, medicine_usages
Breeding	heat_events, inseminations, pregnancy_checks, calvings
Growth	weight_records, fattening_cycles, fattening_costs
Inventory	inventory_items, inventory_batches, stock_movements, stock_transfers
Purchasing	suppliers, purchase_orders, purchase_items, goods_receipts
Sales	customers, sales_invoices, sales_items, payments
Delivery	subscriptions, delivery_routes, delivery_schedules, delivery_records
Accounting	accounts, journal_entries, journal_lines, financial_periods
Employees	employees, attendance, shifts, tasks, payroll
System	notifications, attachments, audit_logs, farm_settings

Design: UUID primary keys, indexed foreign keys, timestamps, farm/organization scope, uniqueness and check constraints, RLS policies, transactional services and immutable posted financial history.

6. Integrity and Accounting Rules

- Unique cattle ID per farm; preserve records after sale or death.
- Atomic inventory movements; prevent unauthorized negative stock.
- Every posted journal must balance debits and credits; correct with reversal entries.
- Reconcile milk production, allocations, collection, sales and stock.
- Separate payments from recognized revenue and inventory purchases from feed consumption costs.
- Avoid double-counting internal branch transfers and animal-level expenses.

7. Version-wise Development Roadmap

Version	Milestone	Deliverables
1.0	Foundation & Security	Architecture, migrations, auth, organization/farm setup, i18n, RLS
1.1	Cattle & Lifetime Records	Registry, IDs, timeline, relationships, transfers
1.2	Milk Production & Collection	Milking, quality, allocation, farmers, settlements
1.3	Feed, Health & Breeding	Rations, treatments, vaccines, pregnancy, calving, growth
1.4	Fattening & Inventory	Cycles, weight gain, stock, purchases, suppliers
1.5	Sales & Distribution	Customers, invoices, subscriptions, routes, returns
1.6	Finance & Accounting	Journals, receivables, payables, statements
1.7	Employees & Reporting	Attendance, payroll, reports, KPIs
1.8	AI & Automation	Grounded assistant, notifications, anomaly detection
2.0	Production & Multi-Branch	Consolidation, security, performance, restore tests, deployment

8. Codex Implementation Workflow

- Read AGENTS.md, README.md, PROJECT_PLAN.md, PROGRESS.md and CHANGELOG.md.
- Inspect existing implementation and select one incomplete milestone.
- Implement its migrations, services, UI, translations, authorization and validation.
- Run unit, integration, end-to-end, lint, typecheck and production-build checks.
- Verify permissions, data integrity and business calculations.
- Update progress, changelog and module documentation; commit verified work.
- Stop and wait for approval before beginning another module.

9. Master Codex Implementation Prompt

You are the lead software architect and senior full-stack developer for FarmLife ERP. Build a production-ready bilingual English/Bangla multi-farm Dairy Management ERP for Bangladesh using React, TypeScript, Vite, Tailwind, shadcn/ui, Supabase PostgreSQL/Auth/Storage/Edge Functions, TanStack Query, Zod, Vitest and Playwright. Inspect the existing repository. Read AGENTS.md, README.md, PROJECT_PLAN.md, PROGRESS.md, CHANGELOG.md and the complete FarmLife ERP Master Build Prompt. Identify completed, incomplete and missing features. Preserve existing code and real records. Follow the approved roadmap; implement ONLY ONE module at a time, including migrations, backend business logic, responsive UI, bilingual translations, RLS, RBAC, validation and tests. Enforce organization/farm isolation and avoid financial or inventory double-counting. Never use demo data as production data or claim tests passed unless executed. After each module, report implemented features, migrations, actual test results, blockers and risks; update documentation and commit verified changes. Push only to an authorized configured remote. STOP and wait for approval before the next module. CURRENT TASK: inspect the repository, reconcile it with the master specification and implement only the next incomplete milestone.

10. Project Documentation & GitHub Backup

File / folder	Purpose
AGENTS.md	Codex operating rules
README.md	Setup and project overview
PROJECT_PLAN.md	Roadmap and dependencies
PROGRESS.md	Verified progress and blockers
CHANGELOG.md	Changes and fixes
DATABASE.md / API.md	Schema, migrations and service contracts
.env.example	Names of required variables; never secrets
docs/modules/	Module-level specifications
supabase/migrations/	Versioned database changes
tests/	Automated tests

11. Acceptance Criteria & Current Status

Each module is complete only when its migration succeeds, workflows operate on real authorized data, English/Bangla UI works, permissions and business rules are tested, and automated checks pass. End-to-end scenarios include cattle lifetime history, milk allocation, farmer collection, inventory consumption, sales, partial payments, balanced journals and consolidated multi-farm reporting.

Current project context: Foundation and Module 1 (Authentication & Farm Setup) implementation archives were prepared earlier, but dependency installation, live Supabase integration and automated verification were not confirmed. Next: reconcile Module 1 with the organization → multiple farms/branches architecture, apply migrations, and verify access isolation before proceeding to Cattle Registry.